

FULL PROJECT REPORT: MULTITHREADED WIKIPEDIA CRAWLER AND FREQUENCY ANALYSER

Course Component: Full-Stack Software Engineering Portfolio

Developer Name: Varri Yaswanth

TECHNOLOGIES USED

Backend

- Java
- Spring MVC

Frontend

- JSP
- HTML
- CSS
- JavaScript

Database

- PostgreSQL
- jOOQ

Libraries

- JSoup
- Gson

Build Tool

- Maven

Deployment

- Jetty Plugin

Concurrency

- Java Multithreading (Runnable Interface)

GitHub Repository

<https://github.com/varriyaswanth2002/wikipedia-crawler>

1. PROJECT SPECIFICATIONS AND ABSTRACT

The **Multithreaded Wikipedia Crawler and Frequency Analyser** is an enterprise-grade Java web platform engineered on top of the **Spring MVC** design architecture. The primary objective of this application is to execute high-volume, concurrent web scraping tasks without inducing server-side thread locking or latency overhead.

The application implements Java's native Runnable concurrency mechanics to instantiate decoupled background workers. These workers concurrently query remote Wikipedia endpoints, extract unstructured textual descriptions, sanitize web content, and isolate absolute media file coordinates.

Additionally, the system incorporates an independent **Word Frequency Analyser** engine. This module processes raw data documents (such as .txt files) using a singleton configuration manager, maps unique string tokens, tracks word distribution matrices, and outputs term densities sorted by occurrence.

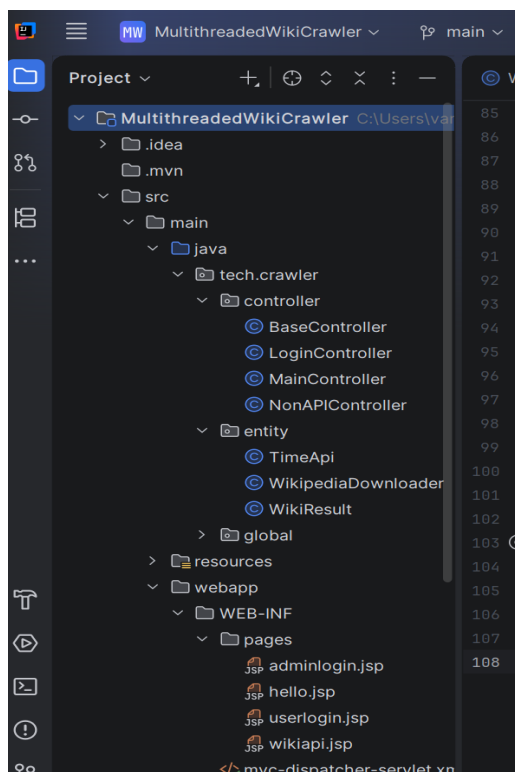
The application provides concurrent Wikipedia crawling based on user keywords, extracts article text and images, generates structured JSON responses, renders dynamic HTML pages, performs word frequency analysis, and stores application data using PostgreSQL with jOOQ.

2. SYSTEM ARCHITECTURE AND FOLDER LAYOUT

The software application follows a standardized Maven compilation directory layout, decoupling backend controllers, database utilities, and client-facing view templates.

2.1 Project Directory Structural Map

The image below captures the absolute folder hierarchy of the application source tree inside the IntelliJ explorer view:



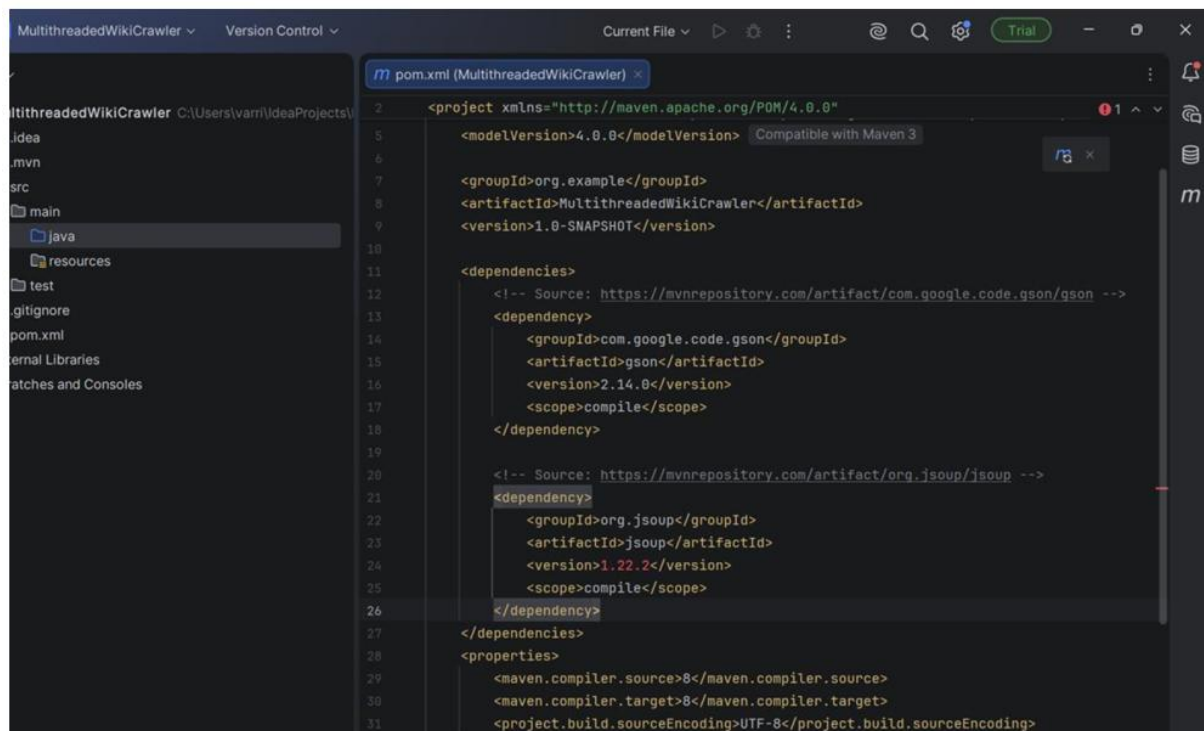
- **tech.crawler.controller:** Directs inbound HTTP request boundaries. Includes BaseController, LoginController, MainController, and NonAPIController to separate data-only REST endpoints from front-facing views.
 - **tech.crawler.entity:** Houses data model representations and execution models, including TimeApi, WikiResult, and the multithreaded core component WikipediaDownloader.
 - **src/main/webapp/WEB-INF/pages/:** Contains client view templates, including adminlogin.jsp, userlogin.jsp, hello.jsp, and wikiapi.jsp.
-

3. DEPENDENCY SELECTION AND BUILD PARAMETERS

The application utilizes Maven to automate software lifecycle processes and maintain external software builds. It integrates **GSON** for rapid object-to-JSON serialization and **JSoup** for parsing remote HTML DOM trees.

3.1 Maven Configuration (pom.xml)

The screenshot below confirms the baseline library specifications used to compile the application modules:



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6     <groupId>org.example</groupId>
7     <artifactId>MultithreadedWikiCrawler</artifactId>
8     <version>1.0-SNAPSHOT</version>
9
10    <dependencies>
11        <!-- Source: https://mvnrepository.com/artifact/com.google.code.gson/gson -->
12        <dependency>
13            <groupId>com.google.code.gson</groupId>
14            <artifactId>gson</artifactId>
15            <version>2.14.0</version>
16            <scope>compile</scope>
17        </dependency>
18
19        <!-- Source: https://mvnrepository.com/artifact/org.jsoup/jsoup -->
20        <dependency>
21            <groupId>org.jsoup</groupId>
22            <artifactId>jsoup</artifactId>
23            <version>1.22.2</version>
24            <scope>compile</scope>
25        </dependency>
26    </dependencies>
27
28    <properties>
29        <maven.compiler.source>8</maven.compiler.source>
30        <maven.compiler.target>8</maven.compiler.target>
31        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
32    </properties>
33 </project>
```

4. MULTITHREADED EXECUTION AND CORE RUNTIME OUTPUTS

4.1 Asynchronous Concurrent Crawling

By overriding the `run()` execution block of the `Runnable` library interface, the `WikipediaDownloader` module launches separate network threads to fetch data for distinct target terms simultaneously:



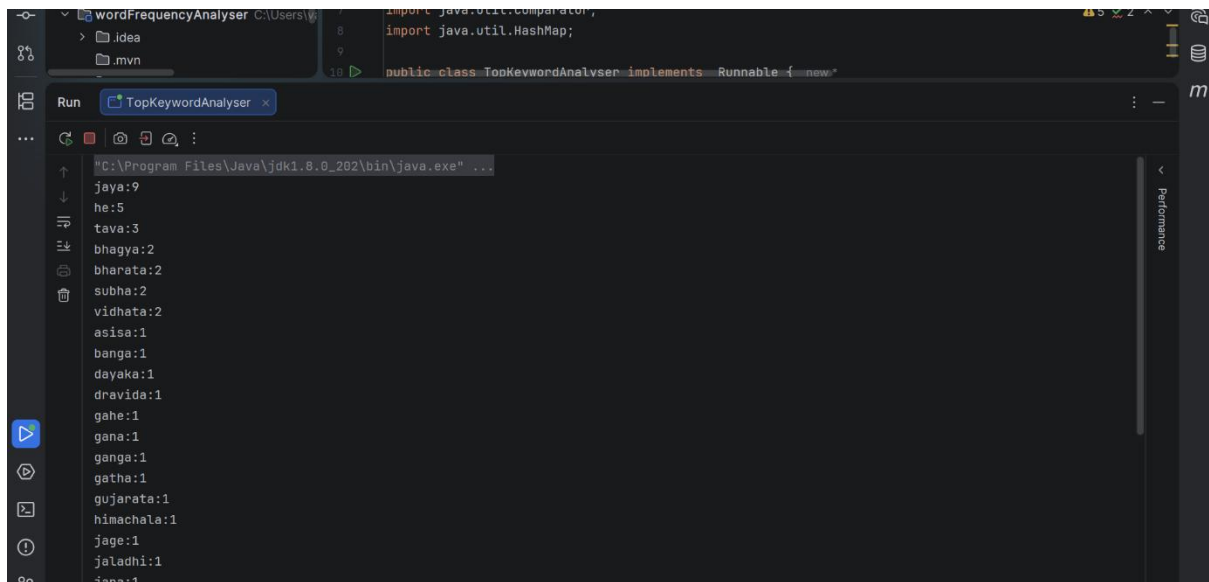
```
public class WikipediaDownloader implements Runnable {
    String arr[] = {"India", "United States"};
    System.out.println("This side is varri yaswanth");
    System.out.println("Running wikipedia downloader at " + new Date().toString());
}

"query": "India",
"text_result": "India, officially the Republic of India,[j][19] is a country in South Asia. It is the seventh-largest country by area, th",
"image_url": "//upload.wikimedia.org/wikipedia/en/thumb/4/41/Flag_of_India.svg/250px-Flag_of_India.svg.png"

"query": "United States",
"text_result": "The United States of America (USA), also known as the United States (U.S.) or America, is a country primarily located in",
"image_url": "//upload.wikimedia.org/wikipedia/en/thumb/a/a4/Flag_of_the_United_States.svg/250px-Flag_of_the_United_States.svg.png"
```

4.2 Word Frequency Density Tracking

The sub-analytical engine reads external file text structures, maps unique character strings, and sorts individual words based on density metrics:



```
import java.util.Comparator;
import java.util.HashMap;

public class TopKeywordAnalyser implements Runnable {
    // ...
}

jaya:9
he:5
tava:3
bhagya:2
bharata:2
subha:2
vidhata:2
asisa:1
banga:1
dayake:1
dravida:1
gahe:1
gana:1
ganga:1
gatha:1
gujarata:1
himachala:1
jage:1
jeladhi:1
jana:1
```

4.3 Database Persistence Node Validation

System parameters, coder log tracking tables, and data nodes are maintained inside a **PostgreSQL** database server context layer using explicit **jOOQ** query layouts:

The screenshot displays a PostgreSQL database management tool interface. The left sidebar shows a tree view of database objects, with 'Codere' expanded under 'Tables (1)'. The main window shows a query editor with the following SQL query:

```
SELECT * FROM public."Codere"
```

The query is executed, and the results are displayed in the 'Data Output' tab. The results show two rows of data:

	name	age
1	varri yaswanth	24
2	Sadie Varri	22

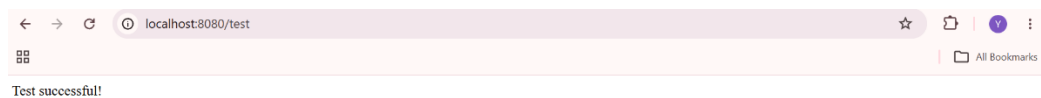
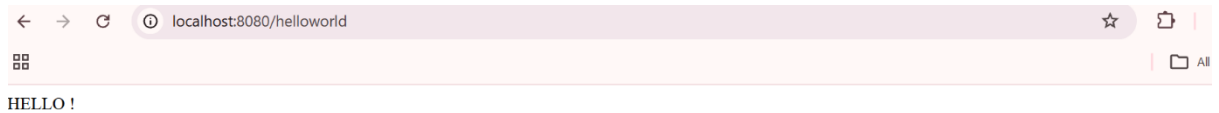
The interface also includes a 'Query History' tab and a 'Scratch Pad' tab. The 'Data Output' tab shows the results of the query, with columns 'name' and 'age'.

5. ENDPOINT ROUTING AND FRONTEND VIEW PROOFS

5.1 Base Gateway Validation

When launching the application context via the Jetty engine plugin, standard health routing paths verify core stability:

- **Endpoint /helloworld:** Confirms basic view resolver mapping.
- **Endpoint /test:** Confirms baseline HTTP connection loops.



5.2 User Interface Portals

The application isolates system administration panels from consumer endpoints using standalone JSP forms:



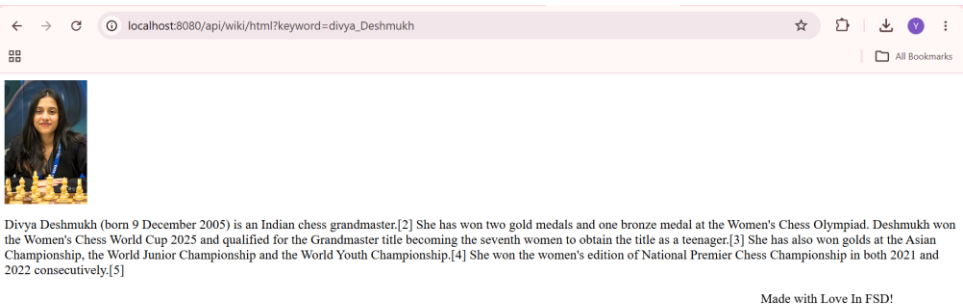
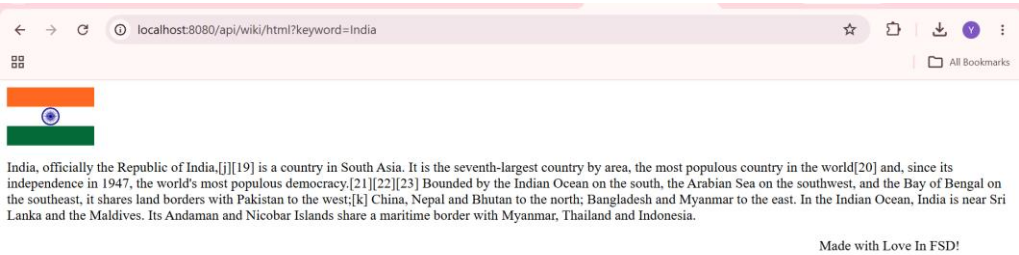
5.3 REST API JSON Payload Conversions

When external components query the REST API context path at /api/wiki?keyword=..., the application returns a raw JSON object string mapping the text summaries and absolute asset urls:



5.4 Dynamic HTML Web Presentations

When a request hits the /api/wiki/html path, the view engine injects the scraped content directly into responsive presentation layers:



Future Enhancements

- Category-based crawling
- Multi-page recursive crawling
- Automatic dataset generation
- Crawl progress monitoring